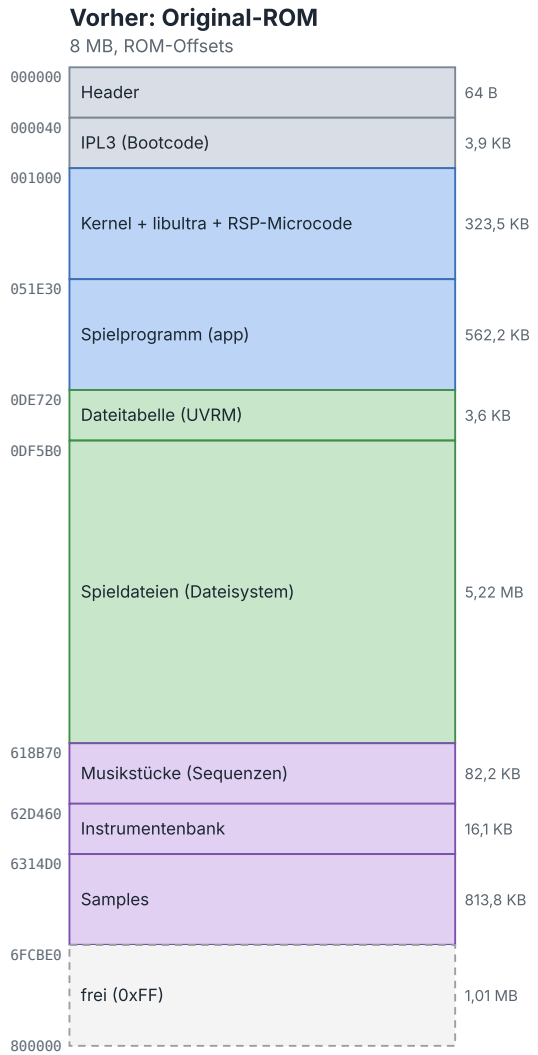
# Pilotwings 64 (USA): Aufteilung von ROM und RAM

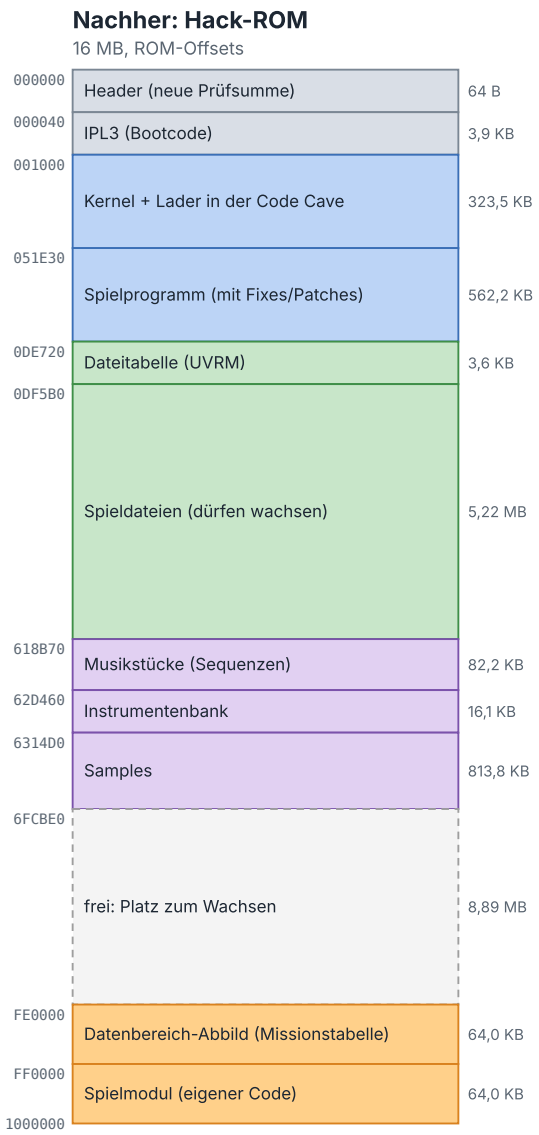
Vorher = Originalspiel (8-MB-ROM, Konsole ohne Expansion Pak). Nachher = Hack-ROM von Hawk's Hangar ab Stufe 3 (16-MB-ROM, Expansion Pak vorausgesetzt). Stand: 15.10.2026.

System / Hardware · Programmcode des Spiels · Spieldaten · Musik und Klänge · Heap (freier Arbeitsspeicher) · Feste Puffer (Bild, Tiefe, Audio) · Ungenutzt (Füllbytes) · Neu durch Hawk's Hangar

## ROM (Steckmodul bzw. .z64-Datei)

Links stehen die Startadressen (Offsets in der ROM-Datei), rechts die Größen. Die Balken sind **nicht maßstabsgetreu**, damit auch die kleinen Abschnitte lesbar bleiben.



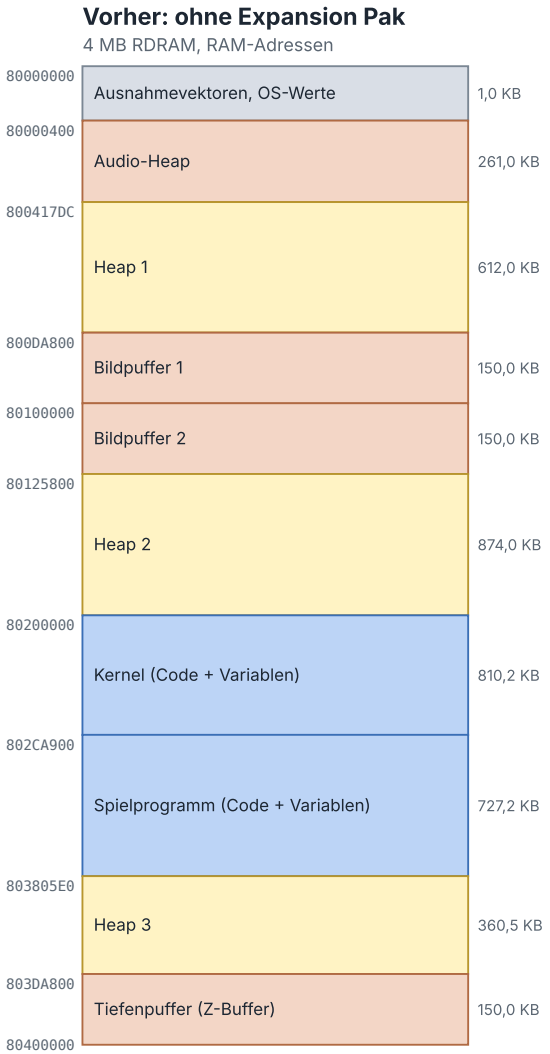

Beim Bau beginnt der Editor immer mit dem sauberen ROM, setzt die Dateien des Projekts ein, legt Musik, Datenbereich und Modul an ihren Platz und berechnet die Boot-Prüfsumme neu. Die Fixes und Patches tauschen einzelne Befehle in Kernel und Spielprogramm aus.

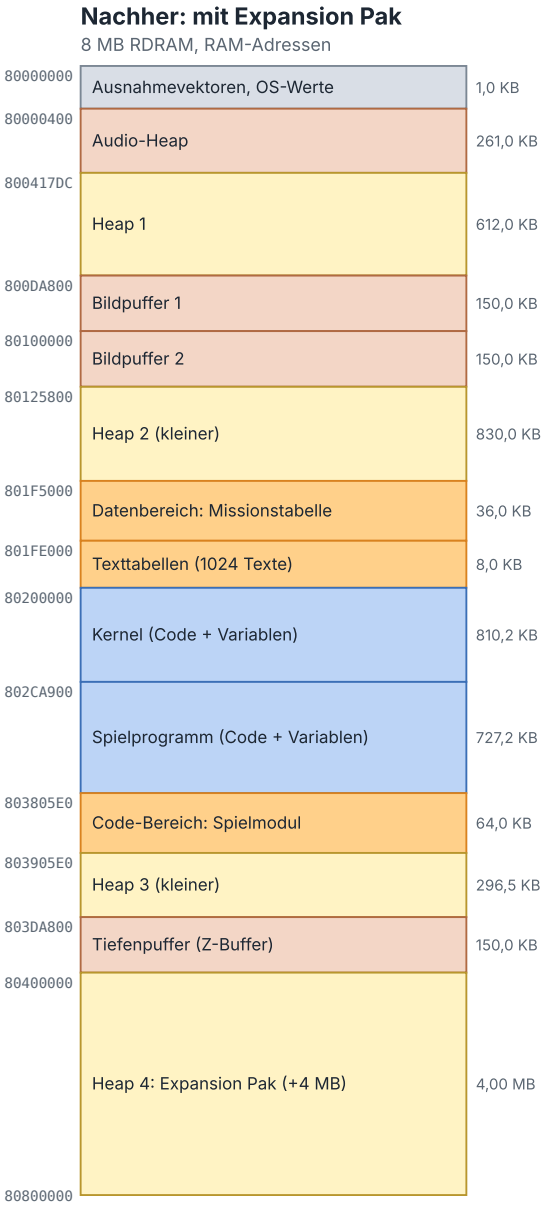
## ROM: die Abschnitte

| | |
|---|---|
| **Header** | Die ersten 64 Bytes: Name des Spiels („Pilot Wings64“), Spielcode NPWE und die Boot-Prüfsumme. Der Editor rechnet die Prüfsumme nach jedem Bau neu aus, weil sich Code im geprüften ersten MB ändert. |
| **IPL3 (Bootcode)** | Startcode für den CIC-Chip (6102). Er prüft das erste MB der ROM und lädt den Anfang des Programms. Bleibt unverändert. |
| **Kernel + libultra + RSP-Microcode** | Das Grundsystem des Spiels („uv“-Funktionen: Speicher, Dateien, Grafik, Ton), Nintendos Bibliothek libultra und die Microcodes für den Grafik-/Tonprozessor. Wird beim Start nach 0x80200000 kopiert. Neu: In der „Code Cave“ (ROM 0x2B4A4–0x2B548, eine Funktion, die sonst nichts tut) liegen die Foto-Helfer und der Lader des Spielmoduls; einzelne Befehle des Speicherverwalters sind geändert (Code-Bereich, Datenbereich, Texttabellen). |
| **Spielprogramm (app)** | Die eigentliche Spiellogik: Fahrzeuge, Missionen, Punktevergabe, Menüs, HUD. Wird nach 0x802CA900 kopiert. Die Code-Fixes und Spiel-Patches tauschen hier einzelne Befehle aus, außerdem schreibt der Editor Tabellen (Ergebniszeilen, Demo-Button). |
| **Dateitabelle (UVRM)** | Liste aller Spieldateien mit Typ und Größe. Das Spiel berechnet die Lage jeder Datei, indem es die Größen aufaddiert. Der Editor schreibt sie neu, wenn sich eine Datei ändert. |
| **Spieldateien** | Alle Inhalte: Inselmodelle, Gelände, Texturen, Missionen (UPWT), Texte, Demoflüge, Kamerapfade usw., viele davon gepackt (MIO0/GZIP). Geänderte Dateien des Projekts werden hier eingesetzt; das Dateisystem darf wachsen. |
| **Musik und Klänge** | Sequenzen (Musikstücke im Compact-MIDI-Format), die Instrumentenbank und die Samples. Ihre Adressen stehen fest im Code; reicht der Platz vor ihnen nicht mehr, verschiebt der Editor den ganzen Block hinter die Dateien und passt die Verweise an. |
| **Frei** | Im Original rund 1 MB Füllbytes (0xFF). Im Hack-ROM sind es rund 8,9 MB: Platz für größere Dateien, neue Inseln und Missionen. Leerer Platz kostet im BPS-Patch fast nichts. |
| **Datenbereich-Abbild** | 64 KB vor dem Modul. Darin liegt die Missionstabelle (Kopf + 256 Bytes je Mission mit Einstellungen: Bedingungen, Treibstoff, harte Endzeit, später mehr). Das Modul kopiert sie beim Start in den Datenbereich des RAM. |
| **Spielmodul** | Die letzten 64 KB der ROM: der eigene Code des Editors (module.c, rund 18 KB). Er prüft das Expansion Pak, wertet Bedingungen, Endzeit und Zielobjekte aus, zeigt Meldungen an und liefert die Debug-Ausgabe. Er wandert immer ans Ende der ROM, egal wie groß sie ist. |

## RAM (Arbeitsspeicher der Konsole)

Links stehen die RAM-Adressen (KSEG0, ab `0x80000000`), rechts die Größen. Die Balken sind **nicht maßstabsgetreu**. Der Speicherverwalter des Spiels kennt eine Liste geschützter Blöcke (Puffer, Programm, Audio); alles dazwischen ist Heap. Neue **Daten** legt der Editor vor den Programmblock (sie wachsen nach unten), neuen **Code** hinter die Variablen des Spiels (er wächst nach oben).



## RAM: die Abschnitte

| | |
|---|---|
| **Ausnahmevektoren, OS-Werte** | Einsprungstellen für Interrupts und Werte des Betriebssystems, z. B. osMemSize (0x80000318: 4 oder 8 MB) und der Fernsehstandard. Hier liest das Modul, ob ein Expansion Pak steckt. |
| **Audio-Heap** | Fester Speicher der Tonausgabe (Abspielzustand, Puffer). |
| **Heap 1–3** | Freier Arbeitsspeicher, aus dem das Spiel beim Laden einer Insel und Mission alles holt: Modelle, Gelände, Texturen, Missionsobjekte, Texte. Ohne Expansion Pak zusammen etwa 1,8 MB. Bei jedem Levelstart wird er gelöscht. Ganz ohne Pak dient ein Teil von Heap 1 (ab 0x80080000) als Bildpuffer für den „EXPANSION PAK REQUIRED“-Bildschirm. |
| **Bildpuffer 1 und 2** | Zwei Bilder zu 320 × 240 Punkten (16 Bit): Eines wird angezeigt, ins andere zeichnet das Spiel das nächste Bild. |
| **Kernel** | Code und Variablen des Grundsystems (kopiert aus der ROM). Der Speicherverwalter schützt diesen Block als „Programmblock“, der jetzt schon bei 0x801F5000 beginnt. |
| **Spielprogramm** | Code und Variablen der Spiellogik bis 0x803805E0. Die alten Texttabellen in diesem Bereich (0x80377F10–0x80378CCC) sind seit expanded-texts-v1 ungenutzt. |
| **Datenbereich** | Neu, 36 KB vor den Texttabellen, dem Heap 2 weggenommen: Kopf + Missionstabelle (bis 128 Missionen, endet bei 0x801FD100), der Rest ist frei. Neue Datentabellen wachsen von hier nach unten (z. B. ein größerer Objektspeicher für Missionen). |
| **Texttabellen** | Seit dem Code-Fix expanded-texts-v1: Namen und Zeiger für bis zu 1024 Texte (vorher 439, im Spielprogramm). |
| **Code-Bereich: Spielmodul** | 64 KB direkt hinter den Variablen des Spiels, dem Heap 3 weggenommen. Der Lader kopiert das Spielmodul vor dem Spielstart hierher. Neuer Code wächst von hier nach oben. |
| **Tiefenpuffer (Z-Buffer)** | Merkt sich pro Bildpunkt die Entfernung, damit Nahes vor Fernem gezeichnet wird. Dient beim Laden auch als Zwischenspeicher. |
| **Heap 4: Expansion Pak** | Die zusätzlichen 4 MB. Das Spiel nutzt sie von selbst als Heap (das Heap-Ende ist osMemSize). Damit ist Platz für größere Inseln, mehr Objekte und neue Tabellen. Jeder Hack setzt das Pak voraus. |

## Ablauf beim Start (nachher)

1. Der IPL3-Bootcode prüft das erste MB und startet den Kernel, der sich nach `0x80200000` kopiert, das Betriebssystem und die Bildausgabe einrichtet.
2. Der Kernel kopiert das Spielprogramm nach `0x802CA900`. Statt es gleich zu starten, ruft er den **Lader in der Code Cave** auf.
3. Der Lader kopiert das Spielmodul vom ROM-Ende nach `0x803805E0` und startet es.
4. Das Modul liest osMemSize. Bei 4 MB zeigt es „EXPANSION PAK REQUIRED“ und hält an. Bei 8 MB kopiert es die Missionstabelle in den Datenbereich (`0x801F5000`) und kehrt zurück.
5. Das Spiel startet wie gewohnt. Sein Speicherverwalter lässt Datenbereich, Texttabellen und Code-Bereich in Ruhe und nutzt die zusätzlichen 4 MB als Heap.